

What this VAE computes

From a transaction row to a high-risk flag

Notes on the `vae-anomaly` implementation

Recorded notebook run at commit `1bae41a`
seed 42, CPU, 25 epochs

Abstract

This note walks one synthetic card transaction through the code in `src/fraudvae/`. It is not a general VAE textbook. Every width below is the width in `TabularVAE`, `VAELoss`, and `ReconstructionScorer`. The fraud label is never an input to training. A row is flagged when its reconstruction score is at least the frozen 95th percentile of a pure-normal calibration set.

1 What you are looking at

The notebook `notebooks/zero_shot_fraud_vae.ipynb` builds 8,000 synthetic rows (7,600 normal, 400 fraud, about 5%), fits a small tabular variational autoencoder on normal rows only, and then scores a mixed validation set. The package split is the same one a later service would keep:

File	Role
<code>dataset.py</code>	synthetic rows, splits, <code>log1p</code> scaler, one-hot indices
<code>model.py</code>	<code>TabularVAE</code> : encoder, reparameterization, branched decoder
<code>loss.py</code>	<code>LossConfig</code> and <code>VAELoss</code>
<code>train.py</code>	<code>VAETrainer</code> , which drops the label
<code>score.py</code>	<code>ReconstructionScorer.score</code>

Defaults used by that notebook, and by every number in this note: latent size 6, hidden width 64, Huber $\delta = 1$, $\lambda_{\text{cont}} = 1$, $\lambda_{\text{cat}} = 1$, $\beta = 0.1$, 25 epochs, Adam learning rate 10^{-3} , batch size 128.

2 Notation

Symbol	Meaning in this repo
$u \in \mathbb{R}_{>0}^5$	Raw continuous fields, in column order: <code>amount</code> , <code>fee</code> , <code>rolling_spend_24h</code> , <code>distance_from_home_km</code> , <code>seconds_since_last_txn</code> .
$c_m \in \{0, \dots, 9\}$	<code>merchant_id</code> index. Token <code>m0</code> is 0, <code>m9</code> is 9.
$c_d \in \{0, 1, 2\}$	<code>device_type</code> index. Order is <code>chip</code> , <code>contactless</code> , <code>online</code> .
$\tilde{u} \in \mathbb{R}^5$	<code>log1p</code> then z-score of u , using mean and std fit on train normals only.
$x \in \mathbb{R}^{18}$	Encoder input: $\tilde{u}$, then the merchant one-hot, then the device one-hot.
$z \in \mathbb{R}^6$	Latent code.
$q_\phi(z x)$	Encoder Gaussian $\mathcal{N}(\mu, \text{diag}(\exp(\log\text{var})))$. ϕ is the encoder weights.
$p(z)$	Prior $\mathcal{N}(0, I_6)$. Not learned.
$p_\theta(x z)$	Decoder. In the writeup this is the generative model. In the loss it is replaced by Huber plus categorical cross-entropy. θ is the decoder weights.
y	Hidden label, 0 normal and 1 fraud. Not an argument of <code>VAELoss</code> .

3 Running example

Two hand-written rows from the notebook. The first sits on the normal generator (fee tracks amount, rolling spend matches a gap of about two hours, merchant `m3`, device `contactless`). The second breaks those links (large amount, fee that does not follow it, long gap with high rolling spend, far from home, merchant `m8`, device `chip`).

Field	On-manifold row	Off-manifold row
<code>amount</code>	24.5	280
<code>fee</code>	0.50	1.2
<code>rolling_spend_24h</code>	57	900
<code>distance_from_home_km</code>	3	80
<code>seconds_since_last_txn</code>	6600	80000
<code>merchant_id</code>	<code>m3</code> (index 3)	<code>m8</code> (index 8)
<code>device_type</code>	<code>contactless</code> (index 1)	<code>chip</code> (index 0)

The scaler printed by the notebook, rounded to three decimals, was

$$m = (3.246, 0.432, 4.068, 1.420, 8.813),$$

$$s = (0.478, 0.179, 0.563, 0.375, 0.504).$$

Those are the training-normal mean and population standard deviation of $\log(1 + u)$, column by column. Applying them at that printed precision gives

$$\tilde{u}_{\text{on}} \approx (-0.02, -0.15, -0.01, -0.09, -0.04),$$

$$\tilde{u}_{\text{off}} \approx (5.01, 1.99, 4.86, 7.93, 4.91).$$

The on-manifold row is near the origin of the scaled space. The off-manifold row is several training standard deviations out on every continuous column, and about eight on distance. That gap is built into the synthetic generator. Section 13 says what that implies for the perfect fraud recall.

4 From a raw row to an 18-vector

`ContinuousPreprocessor` in `dataset.py` does two steps and then stops.

1. Reject the row if any continuous value is non-finite or not strictly positive. Zero is rejected. There is no clamp.
2. $\tilde{u}_j = (\log(1 + u_j) - m_j)/s_j$ for $j = 1, \dots, 5$.

`fit` runs only on the 5,200 normal training rows. `TransactionDataModule` then calls `transform` on calibration and validation. `ReconstructionScorer` calls the same `transform`. Refitting on fraud, on the validation mix, or inside `score` would be a different model.

Categorical strings become indices in `encode_categoricals`. The vocab order is the class order:

- `merchant_id`: m0, m1, ..., m9.
- `device_type`: chip, contactless, online.

An unknown token raises. It is not mapped to a zero vector.

`TabularVAE.encode` builds the one-hots and concatenates. For the on-manifold example, $c_m = 3$ and $c_d = 1$, so the 18-vector is

$$x = [\tilde{u} \mid e_3^{(10)} \mid e_1^{(3)}] \in \mathbb{R}^{18},$$

where $e_3^{(10)}$ is the length-10 vector with a 1 in position 3 (0-based) and $e_1^{(3)}$ is (0, 1, 0). Five continuous coordinates, ten merchant coordinates, three device coordinates. That 18 is $5 + 10 + 3$, not a tuned hyperparameter.

5 Encoder: 18 to 64 to 64 to two vectors of 6

`TabularVAE.encoder` is two linear layers with a ReLU after each:

$$\begin{aligned} h_1 &= \text{ReLU}(W_1 x + b_1), & W_1 &\in \mathbb{R}^{64 \times 18}, \\ h_2 &= \text{ReLU}(W_2 h_1 + b_2), & W_2 &\in \mathbb{R}^{64 \times 64}. \end{aligned}$$

Two more linear maps, with no activation, read h_2 :

$$\mu = W_\mu h_2 + b_\mu, \quad \text{logvar} = \text{clamp}(W_\lambda h_2 + b_\lambda, -10, 10),$$

both in $\mathbb{R}^6$. The clamp is only there so $\exp(\text{logvar})$ cannot overflow. A healthy run stays far inside $[-10, 10]$.

This defines the approximate posterior

$$q_\phi(z \mid x) = \mathcal{N}(\mu, \text{diag}(\exp(\text{logvar}))).$$

The diagonal has six entries. There is no covariance between latent coordinates. ϕ means every encoder weight above, including W_μ and W_λ .

6 Why the marginal likelihood is intractable

The generative story the math starts from is

$$z \sim p(z) = \mathcal{N}(0, I_6), \quad x \sim p_\theta(x | z).$$

The quantity we would like, for a single observed row, is the marginal

$$p_\theta(x) = \int p_\theta(x | z) p(z) dz.$$

That integral is six-dimensional. $p_\theta(x | z)$ is a neural net, not a conjugate density, so there is no closed form. A junior-engineer reading of “intractable” here is narrow and concrete: we cannot evaluate or differentiate $\log p_\theta(x)$ by summing a finite table, because z is continuous and the decoder is a stack of ReLUs. The evidence lower bound replaces that integral with something the code can differentiate.

7 ELBO, then the loss this repo actually minimizes

Start from the marginal and insert the encoder density, which integrates to one:

$$\log p_\theta(x) = \log \int q_\phi(z | x) \frac{p_\theta(x | z) p(z)}{q_\phi(z | x)} dz = \log \mathbb{E}_{q_\phi(z|x)} \left[\frac{p_\theta(x | z) p(z)}{q_\phi(z | x)} \right].$$

$\log$ is concave, so Jensen’s inequality moves it inside the expectation and produces a lower bound:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)).$$

The right-hand side is $\text{ELBO}(x)$. Training would ideally maximize it, which is the same as minimizing its negative.

Read the two pieces the way the optimizer feels them.

- The expectation is the reconstruction term. For the z the encoder just proposed, the decoder has to assign high probability to the row that was actually observed: the five scaled continuous values and the two class indices.
- The KL term keeps $q_\phi(z | x)$ near $\mathcal{N}(0, I_6)$. Without it the encoder can give every row its own far-away code and the decoder can memorize those codes one by one. With it, normal rows have to share one region of $\mathbb{R}^6$, which is the region the decoder learns to read.

VAELoss does *not* implement $-\text{ELBO}$ with a single likelihood. The continuous head has no variance output, and the continuous penalty is Huber loss rather than a Gaussian log-density. Cross-entropy on the categorical heads *is* a negative log-likelihood, for a categorical distribution whose logits the decoder emits. The KL term *is* the exact closed form for a diagonal Gaussian against $\mathcal{N}(0, I_6)$, then multiplied by $\beta = 0.1$ instead of the ELBO’s implicit weight of 1.

So the implemented objective is a practical mixed-data surrogate for the negative expected log-likelihood, plus a downweighted KL. Huber plus cross-entropy stand in for $-\mathbb{E}[\log p_\theta(x | z)]$. They are not themselves one joint density evaluated at x .

8 Reparameterization, and why scoring skips it

The reconstruction term has to be estimated by a sample from $q_\phi(z \mid x)$, and that sample still has to carry a gradient into μ and logvar. `TabularVAE.reparameterize` writes the sample as a deterministic function of a parameter-free noise variable:

$$\sigma = \exp(0.5 \cdot \text{logvar}), \quad \epsilon \sim \mathcal{N}(0, I_6), \quad z = \mu + \sigma \odot \epsilon.$$

$\odot$ is elementwise. `forward` is the training path: encode, draw z , decode. The KL term does not need ϵ . It is computed from μ and logvar in closed form. The sample exists so the reconstruction term can push on the encoder.

`ReconstructionScorer` does not call `reparameterize`. It encodes, ignores logvar for the draw, and calls `decode(mu)`. The score is then a function of the row and the weights only. A fresh ϵ on every request would move the score and make the frozen threshold a moving target.

9 Decoder: 6 to 64 to 64, then three heads

`decode` maps $z \in \mathbb{R}^6$ through the same shape of trunk as the encoder, pointed the other way:

$$\begin{aligned} g_1 &= \text{ReLU}(W_3 z + b_3), & W_3 &\in \mathbb{R}^{64 \times 6}, \\ g_2 &= \text{ReLU}(W_4 g_1 + b_4), & W_4 &\in \mathbb{R}^{64 \times 64}. \end{aligned}$$

Three linear heads read g_2 . None of them has a softmax or a sigmoid.

Head	Map	Output
<code>continuous_head</code>	$\mathbb{R}^{64} \rightarrow \mathbb{R}^5$	scaled amount, fee, rolling_spend_24h, distance_from_home_km, seconds_since_last_txn
merchant head	$\mathbb{R}^{64} \rightarrow \mathbb{R}^{10}$	raw logits
device head	$\mathbb{R}^{64} \rightarrow \mathbb{R}^3$	raw logits

`F.cross_entropy` applies the log-softmax. Putting a softmax in the module would feed probabilities into that function and break the loss. The continuous head is linear because its targets are z-scores: on normal rows they sit near 0 and are unbounded in principle.

10 The batch loss, exactly

For a batch of B rows, `VAELoss.forward` returns four scalars: `total`, `huber`, `cross_entropy`, `kl`. The three components are unweighted. `total` applies `LossConfig`.

Huber. `F.huber_loss` with $\delta = 1$ and `reduction="mean"`. That mean runs over the batch and over the five continuous columns, so each of the $5B$ residuals counts equally. With $\delta = 1$ in the already standardized space, the kink is one training standard deviation of normal $\log(1 + u)$ traffic. Inside the kink the penalty is quadratic. Outside it the slope is constant:

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq 1, \\ |r| - \frac{1}{2} & |r| > 1. \end{cases}$$

Squared error would keep growing with r^2 . A few legitimately large normal amounts, even after `log1p`, would dominate the gradient. Huber still penalizes them. The slope stays bounded.

Cross-entropy. For each head, `F.cross_entropy` with `reduction="mean"` averages over the batch only. The two head-means are then summed. Merchant and device each contribute. Because a sum of means is the mean of the sum, this is also the batch mean of $(\text{CE}_{\text{merchant}} + \text{CE}_{\text{device}})$ on each row. The class target is the integer index, not a one-hot.

KL. For row i and latent coordinate $j = 1, \dots, 6$,

$$\text{KL}_i = -\frac{1}{2} \sum_{j=1}^6 \left(1 + \log \text{var}_{i,j} - \mu_{i,j}^2 - \exp(\log \text{var}_{i,j}) \right), \quad \text{KL} = \frac{1}{B} \sum_{i=1}^B \text{KL}_i.$$

Sum over the six latent dimensions, then mean over the batch. A sum over the batch would grow when the batch grew, and the loss would not be comparable across the last short batch. `VAETrainer` reweights epoch averages by batch size for that reason.

Total.

$$\text{total} = 1 \cdot \text{Huber} + 1 \cdot (\text{CE}_{\text{merchant}} + \text{CE}_{\text{device}}) + 0.1 \cdot \text{KL}.$$

$\beta = 0.1$ rather than 1 so the KL term does not take the whole objective while the decoder is still learning the normal manifold. At epoch 1 of the recorded run the unweighted pieces were about Huber 0.422 and cross-entropy 3.270 (near $\log 10 + \log 3$) against KL 0.048. The reconstruction terms did fall. By epoch 25 the printed line was

$$\text{total} = 0.8791, \quad \text{Huber} = 0.2116, \quad \text{CE} = 0.0861, \quad \text{KL} = 5.8136.$$

After the weights, that is Huber 0.2116, cross-entropy 0.0861, and $\beta \cdot \text{KL} = 0.5814$. The weighted KL term was larger than Huber and cross-entropy together (0.2977). That is a fact about this run, not a reason to put KL into the anomaly score. The reconstruction terms had already dropped. KL stayed a training regularizer. Section 12 leaves it out of the score on purpose.

11 Why normal-only training is the detector

`generate_synthetic_transactions` draws both classes. `TransactionDataModule` then cuts them apart. The notebook split of the 7,600 normals is 5,200 train, 1,200 calibration, 1,200 validation. All 400 fraud rows go to validation. The module checks that train and calibration are entirely label 0, and that no `row_id` is shared.

`VAETrainer.fit` unpacks `(continuous, categorical, labels)` and does not pass `labels` into the model or the loss. ϕ and θ are fit only to normal traffic: fee tracks amount, rolling 24h spend falls as the gap grows, device follows distance, merchant follows spend. Those links are the generator in `dataset.py`, documented there so the synthetic shift is not a surprise.

A fraud row still has an encoder and a decoder. The encoder proposes a z . The decoder emits the reconstruction it learned from normals. Huber and cross-entropy come out higher because that reconstruction is a normal-like row and the inputs are not. The label is used only afterward, in `detection_report`, to compute precision, recall, and F1. Nothing in the training loop classifies.

12 Per-row score and the frozen threshold

`ReconstructionScorer.reconstruction_errors` uses a different reduction from `VAELoss`, on purpose. Training logged means so the optimizer saw a stable scale. The threshold lives on one number per row.

For a single row, with $\delta = 1$:

$$\begin{aligned}\text{Huber}_{\text{sum}} &= \sum_{j=1}^5 \ell_1(\hat{u}_j - \tilde{u}_j), \\ \text{CE}_{\text{sum}} &= \text{CE}(\text{logits}_{\text{m}}, c_{\text{m}}) + \text{CE}(\text{logits}_{\text{d}}, c_{\text{d}}), \\ \text{score} &= \lambda_{\text{cont}} \cdot \text{Huber}_{\text{sum}} + \lambda_{\text{cat}} \cdot \text{CE}_{\text{sum}}.\end{aligned}$$

With the default weights the score is the sum of five Huber residuals and two cross-entropies. KL is not added. `decode` is called on μ , not on a sample.

`fit_threshold` runs that score on the 1,200 pure-normal calibration rows and stores the 95th percentile. In the recorded run the notebook printed

threshold = 2.1162

(the full float on the scorer was 2.116162061691284). Those 1,200 rows were not used to fit ϕ or θ , and they are not the rows the classification report uses. Later calls only read the stored float. `score` does not recompute a percentile.

`high_risk` is true when `score` $\geq$ threshold. The comparison is greater-than-or-equal. On the dataframe path the flag text is `High-Risk Anomaly`.

`score` returns five fields: `reconstruction_error`, `threshold`, `high_risk`, `continuous_error`, and `categorical_error` (a dict with `merchant_id` and `device_type`). `score_batch` is the same function over a list.

On the two running examples the notebook printed approximately 0.052 (not high-risk) and 25.95 (high-risk), both against threshold 2.1162.

KL stays out of the score because it answers a different question: how far $q_\phi(z \mid x)$ sits from $\mathcal{N}(0, I_6)$, not whether the row was rebuilt. By epoch 25 that term already outweighed reconstruction in the training objective. Adding it would rank rows by posterior geometry, including rows the decoder reconstructed cleanly, and the calibration percentile would no longer be a percentile of reconstruction error.

13 What the validation numbers do and do not say

The mixed validation set is 1,200 held-out normals plus all 400 fraud rows. `sklearn` classification report from that run, positive class fraud:

Class	Precision	Recall	F1	Support
Normal	1.000	0.956	0.977	1200
Fraud	0.883	1.000	0.938	400

Accuracy was 0.967. The flag rate was 0.044 on validation normals and 1.000 on validation fraud. A score with no class signal, thresholded at the 95th percentile, would flag about 5% of fraud. Recall 1.0 beats that chance baseline by a wide margin *on this generator*.

It is a sanity check, not a production operating point. Fraud rows were drawn to be jointly unlikely under the normal process: shifted continuous margins and broken links, not a relabeling of ordinary rows. On the validation set the mean scaled normal vector was about zero, while the mean scaled fraud vector was about (4.9, 2.3, 4.8, 7.9, 4.7). Fraud reconstruction scores ran from about 13.2 to 30.9, all above 2.1162. Normal validation scores had a 95th percentile near 2.04,

so a few normals still cross the calibration threshold. Those are the false positives behind fraud precision 0.883.

If a later table of real transactions produces a weak F1, the useful reading is that the generator or the loss balance changed, not that this recall transfers to a book of card payments.

14 Pipeline

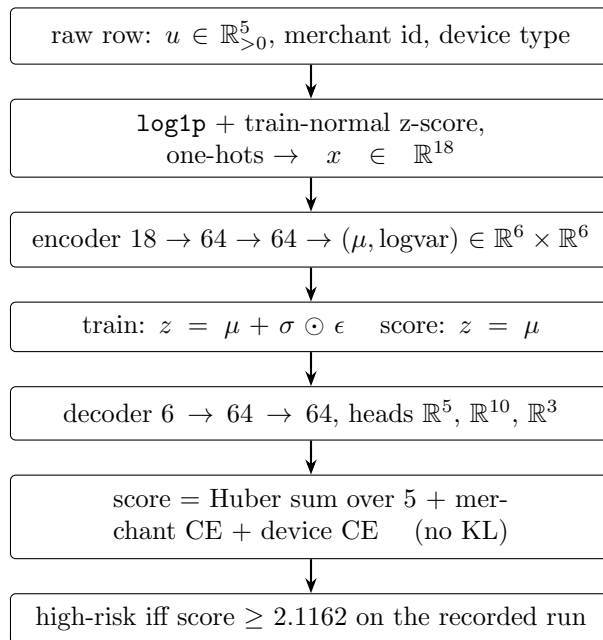


Figure 1: One row through `TabularVAE` and `ReconstructionScorer`. The threshold is data from the calibration split, not a layer.

15 Common misconceptions

1. **This is not a supervised fraud classifier.** `VAETrainer` drops y . There is no sigmoid over “fraud” and no cross-entropy against the label. The label appears only in `detection_report`, after scores exist.
2. **The decoder does not output a fraud probability.** It outputs five scaled continuous numbers and two logit vectors, of lengths 10 and 3. A softmax is not even applied in `model.py`.
3. **The reconstruction score is not a calibrated probability of fraud.** It is a sum of Huber residuals and cross-entropies in the scaled space. Threshold 2.1162 means “at or above the 95th percentile of 1,200 normal calibration rows in this run.” It does not mean “probability 0.95.”
4. **KL is a training regularizer, not part of the score.** It shapes $q_\phi(z \mid x)$ while the weights are being fit. At epoch 25 its weighted value was the largest training term. `reconstruction_errors` never adds it.

5. **The threshold is not recomputed per request**, and it is not fit on the rows you report metrics on. `fit_threshold` refuses a frame that contains a fraud label.
6. **One-hot order is part of the model**. Merchant block, then device block, class index equal to vocab position. A swapped vocab is a different network than the one that was trained.

16 Trace through the code

Follow the on-manifold example from the notebook cell that calls `score`.

1. `dataset.py`: `generate_synthetic_transactions` builds the table. `TransactionDataModule` splits it, fits `ContinuousPreprocessor` on train normals, and yields (`continuous`, `categorical`, `labels`) from `TransactionDataset`. The training loader is normal rows only.
2. `model.py`: `TabularVAE.forward` calls `encode` (concat to 18, trunk to h_2 , then μ and `logvar`), `reparameterize`, and `decode` (trunk from z , then the 5-, 10-, and 3-wide heads).
3. `loss.py`: `VAELoss.forward` computes the mean Huber, the summed per-head mean cross-entropies, the batch-mean KL, and `total` with λ_{cont} , λ_{cat} , and β from `LossConfig`.
4. `train.py`: `set_seed(42)` seeds Python, NumPy, and PyTorch. `VAETrainer.fit` runs Adam for 25 epochs and prints the four scalars. The label tensor is bound to `_labels` and unused.
5. `score.py`: `ReconstructionScorer.fit_threshold` on the calibration frame stores 2.1162. `score` builds a one-row frame, reuses `reconstruction_errors` (`preprocess`, `encode`, `decode(mu)`, per-row sum), and sets `high_risk` by comparing to the stored threshold. `score_batch` is that path over a list.
6. The notebook wires those calls in order: `data`, `TabularVAE`, `VAELoss`, `VAETrainer`, then `ReconstructionScorer`. The shape check before training decodes μ and does not draw ϵ , so it does not consume the reparameterization noise the trainer is about to use.